LP ANSWERS CHAPTER-2

1. Explain memory mapping of singly linked list with neat diagram.

Answer:

A **Singly Linked List (SLL)** is a data structure consisting of **nodes**, where each node contains:

1. **Data** – stores the value
2. **Pointer (next)** – stores the address of the next node in memory

Nodes are stored in **non-contiguous memory locations**, and each node points to the next node using a pointer.

Head

```
|

+-------+--------+     +-------+--------+     +-------+--------+

| Data  | Next   | --> | Data  | Next   | --> | Data  | Next   |

|  10   | 0x102  |     |  20   | 0x103  |     |  30   | NULL   |

+-------+--------+     +-------+--------+     +-------+--------+
```

2. Compare linked list with arrays?

Answer:

Feature	Array	Linked List (Singly)
Memory Allocation	Static (fixed size, allocated at compile-time)	Dynamic (size can grow or shrink at runtime)
Memory Location	Contiguous memory locations	Non-contiguous memory locations
Size	Fixed (must know in advance)	Flexible / dynamic
Access Time	Direct access using index, O(1)	Sequential access required, O(n) to access ith element
Insertion/Deletion	Costly if not at end, O(n) (requires	Efficient anywhere, O(1) if pointer to

Feature	Array	Linked List (Singly)
	shifting)	node is known
Memory Wastage	May waste memory if allocated size is larger than needed	No pre-allocation waste; memory used as needed
Pointer Usage	No pointers needed	Uses pointers to link nodes
Cache Friendliness	Good (contiguous memory)	Poor (nodes scattered in memory)
Use Cases	When size is known and frequent access by index is required	When size is unknown or frequent insertions/deletions needed

3. Mention two real-world applications of linked lists and explain two major advantages of doubly linked lists over singly linked lists.

Answer:

Two Real-World Applications of Linked Lists

1. **Music Playlist Management**
   - Songs are stored as nodes in a linked list.
   - Allows easy insertion or deletion of songs anywhere in the playlist without shifting data.
2. **Web Browser History / Forward-Backward Navigation**
   - Each visited page is a node in a linked list.
   - Moving back and forth between pages is easier using pointers rather than arrays.

Two Major Advantages of Doubly Linked Lists over Singly Linked Lists

1. **Bidirectional Traversal**
   - Doubly linked lists have **two pointers per node** (`next` and `prev`)
   - You can traverse **forward and backward**, unlike singly linked lists which only allow forward traversal.
2. **Efficient Deletion/Insertion**
   - In a doubly linked list, a node can be deleted or inserted **without knowing the previous node** in advance.
   - In singly linked lists, deletion requires **traversing from head** to find the previous node.

4. A railway reservation system keeps a waiting list of passengers. New passengers are added to the end of the list, and when a seat becomes available, the passenger who has been waiting the longest is removed from the list and allotted the seat. Identify the appropriate data structure for this system and specify the type of insertion and deletion operations involved.

Answer:

Data Structure

- **Singly Linked List (SLL)**
- Each node represents a **passenger** and contains:
  1. Passenger data (name, ID, etc.)
  2. `next` pointer to the next passenger
- **Head** → points to the **front** of the waiting list

Operations Using SLL

Operation	Implementation in SLL
Insertion (New Passenger)	Add node at the **end** of the list (traverse to last node, then `next = newNode`)
Deletion (Seat Available)	Remove node from the **head** of the list (`head = head->next`)

Step-by-Step

1. **Enqueue (Add at rear)**

```
head -> Node1 -> Node2 -> Node3 -> NULL
Add new passenger (Node4) at the end:
head -> Node1 -> Node2 -> Node3 -> Node4 -> NULL
```

2. **Dequeue (Remove from front)**

```
head -> Node1 -> Node2 -> Node3 -> Node4 -> NULL
Seat available → remove Node1:
head -> Node2 -> Node3 -> Node4 -> NULL
```

5. An operating system maintains a list of active processes, where each process is identified by its process ID and priority. The processes are executed in order of priority, from highest to lowest. New processes can be inserted at appropriate positions in the list based on their priority. Once a process completes its execution, it is removed from the list of active processes. Identify the most suitable data structure for managing this system and explain what type of the insertion and deletion operations involved.

Answer:

Scenario Using SLL

- **Each node** stores:
  - Process ID
  - Priority
  - Pointer to next node
- **Head node** points to the **highest priority process**.

Operations Using SLL

Operation	Implementation
Insertion (new process)	Traverse the SLL from head. Insert the new process **before the first node that has lower priority** than the new process. This keeps the list sorted by priority (highest first).
Deletion (process completes)	Remove the **head node** (always the process with the highest priority).

Step-by-Step Example

1. Current list (priority): `P1(50) -> P2(30) -> P3(20) -> NULL`
2. New process `P4(40)` arrives:
   - Traverse list → insert **between 50 and 30**
   - New list: `P1(50) -> P4(40) -> P2(30) -> P3(20) -> NULL`
3. Highest priority process completes → delete head:
   - Remove `P1(50)`
   - List: `P4(40) -> P2(30) -> P3(20) -> NULL`

6. Solved in Class Present in Notes

7. A bus service management system maintains buses in a circular route sequence so that after the last bus completes the route, the sequence continues again from the first bus. Each bus stores details such as Bus ID, Bus Number, and Trip Duration. Write a modular C program to manage this route sequence by performing the following operations: add a new bus at the end of the sequence, remove the last bus from the sequence, display the details of all buses in circular order, and display the details of the bus having the highest trip duration.

Answer:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Structure for a bus
struct Bus {
    int busID;
    char busNumber[20];
    float tripDuration;
    struct Bus *next;
};

// Function to add a bus at the end
struct Bus* addBusEnd(struct Bus *head) {
    struct Bus *newBus = (struct Bus*)malloc(sizeof(struct
Bus));

    printf("Enter Bus ID: ");
    scanf("%d", &newBus->busID);
    printf("Enter Bus Number: ");
    scanf("%s", newBus->busNumber);
    printf("Enter Trip Duration: ");
    scanf("%f", &newBus->tripDuration);
    newBus->next = NULL;
```

```
    if (head == NULL) {  // empty list
        newBus->next = newBus;
        head = newBus;
    } else {
        struct Bus *temp = head;
        while(temp->next != head) {   // traverse to last
node
            temp = temp->next;
        }
        temp->next = newBus;
        newBus->next = head;  // maintain circular link
    }
    return head;
}

// Function to remove last bus
struct Bus* removeLastBus(struct Bus *head) {
    if (head == NULL) {
        printf("No buses to remove.\n");
        return NULL;
    }

    if (head->next == head) {  // only one bus
        free(head);
        return NULL;
    }

    struct Bus *temp = head;
    struct Bus *prev = NULL;
    while (temp->next != head) {  // traverse to last node
        prev = temp;
        temp = temp->next;
    }
    prev->next = head;  // second last node points to head
```

```
    free(temp);
    return head;
}

// Function to display all buses
void displayBuses(struct Bus *head) {
    if (head == NULL) {
        printf("No buses in the route.\n");
        return;
    }

    struct Bus *temp = head;
    printf("\nBus Details in Circular Route:\n");
    do {
        printf("Bus ID: %d, Bus Number: %s, Trip Duration:
%.2f\n",
               temp->busID,      temp->busNumber,      temp-
>tripDuration);
        temp = temp->next;
    } while(temp != head);
}

// Function to display bus with highest trip duration
void displayMaxDurationBus(struct Bus *head) {
    if (head == NULL) {
        printf("No buses in the route.\n");
        return;
    }

    struct Bus *temp = head;
    struct Bus *maxBus = head;

    do {
        if (temp->tripDuration > maxBus->tripDuration)
            maxBus = temp;
```

```
        temp = temp->next;
    } while(temp != head);

    printf("\nBus with Highest Trip Duration:\n");
    printf("Bus ID: %d, Bus Number: %s, Trip Duration: %.2f\n",
           maxBus->busID,    maxBus->busNumber,    maxBus->tripDuration);
}

int main() {
    struct Bus *head = NULL;
    int choice;

    do {
        printf("\nBus Route Management:\n");
        printf("1. Add Bus at End\n2. Remove Last Bus\n3. Display All Buses\n4. Display Bus with Highest Trip Duration\n5. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch(choice) {
            case 1: head = addBusEnd(head); break;
            case 2: head = removeLastBus(head); break;
            case 3: displayBuses(head); break;
            case 4: displayMaxDurationBus(head); break;
            case 5: printf("Exiting program.\n"); break;
            default: printf("Invalid choice.\n");
        }
    } while(choice != 5);

    return 0;
}
```

8. **Solved in Class Present in Notes**
9. **Home Work**